ARDUINO WORKSHOP SERIES

Three standalone, hands-on sessions for grades 5 to 8: a traffic light, a reaction time game, and a servo creature. Everything a team or teacher needs to run them.

GRADES	SESSIONS	GROUP SIZE	EXPERIENCE
5 to 8	3, about 55 to 70 min each	Any size, students work in pairs	None needed, for students or teachers

This is a three session Arduino mini-course for middle schoolers. Every session ends with something that lights up, beeps, or moves, so students see results within the first half hour. Along the way they practice the habit that matters most in engineering: wire it, upload it, test it, find what's wrong, and make it better.

Each session stands on its own. A student who only shows up for session 2 can still finish the reaction game. Students who come to all three see the bigger idea: hardware plus code makes interactive systems, and the same few building blocks (pins, inputs, outputs, timing) show up in everything from a crosswalk signal to a robot arm.

WE PROVIDE

- Every part for every pair: Arduino boards, USB cables, breadboards, LEDs, buttons, buzzers, servos, and wires (about $200 in parts)
- Lesson plans for all three sessions, with code you can copy
- A training call for your teachers or team. No Arduino experience needed
- A Spark Gears mentor on call while you run it

THE HOST PROVIDES

- A room with tables and power outlets
- The students, plus sign-ups and marketing
- One or two adults or team members who attend the training
- About an hour before the first session to set up

STUDENTS BRING

- A laptop: personal or school issued
- Chromebooks work through the free Arduino Cloud Editor
- Windows or Mac laptops can use the Arduino IDE

WHAT STUDENTS WALK AWAY WITH

SESSION	THEY BUILD	THEY LEARN
1. Traffic light	Three LEDs that cycle green, yellow, red	Digital outputs, resistors, breadboard rows, delay()
2. Reaction game	A game that times how fast you press a button	Inputs, INPUT_PULLUP, random(), millis(), tone()
3. Servo creature	A cardboard creature that nods, flaps, or chomps	Servo control, analog input, map(), designing behavior

BEFORE SESSION 1

Do this a few days ahead. Most first-day problems are laptop setup, not wiring.

1. Test one laptop of every type students will bring. On Chromebooks, open app.arduino.cc, make a free account, and install the Arduino Cloud Agent when it asks. On Windows or Mac, install Arduino IDE 2 from arduino.cc/en/software.
2. Upload the built-in Blink example (File, Examples, 01.Basics, Blink) to a board. If the little on-board LED blinks, that laptop type is ready.
3. Count out parts into one bag per pair. Label the bags so cleanup is fast.
4. If students are under 13, check how your school handles Arduino accounts. Some schools have teachers share one account across the room.

TIP FROM OUR FIRST WORKSHOPS

Have one spare laptop already set up and tested. When a student's computer refuses to see the board, swap it in and keep them building instead of debugging drivers in front of the class.

SESSION 1 | 55 MINUTES

TRAFFIC LIGHT

Students wire red, yellow, and green LEDs on a breadboard and program them to run like a real traffic light. It's the simplest possible project that still feels like a real thing, and it teaches the two ideas every later session depends on: a pin can turn something on and off, and code runs one line at a time, in order.

PARTS PER PAIR

PART	QTY	NOTES
Arduino Uno (or compatible) + USB cable	1	Check the cable carries data, not just power
Half-size breadboard	1	
LEDs: red, yellow, green	1 each	Long leg is positive (+)
220 ohm resistors	3	Red, red, brown bands
Jumper wires (male to male)	6 to 8	Extras for the challenge

RUN OF SESSION

TIME	WHAT HAPPENS	TEACHING NOTES
0 to 8 min	Plug in the Arduino, pick the board and port, upload Blink	Walk the room. Every pair should see the on-board LED blink before moving on.
8 to 25 min	Wire the three LEDs	Show how breadboard rows connect. Each LED: pin to resistor to long leg, short leg to GND.
25 to 50 min	Code the light sequence and tune the timing	Start with just green, then add yellow and red. Let them pick their own timing.
50 to 55 min	Show one or two working lights, then clean up	Parts back in the bag, count the LEDs.

WIRING

LED	ARDUINO PIN	PATH
Red	8	Pin 8 to 220 ohm resistor to LED long leg. Short leg to GND rail.
Yellow	9	Pin 9 to 220 ohm resistor to LED long leg. Short leg to GND rail.
Green	10	Pin 10 to 220 ohm resistor to LED long leg. Short leg to GND rail.
GND rail	GND	One wire from any GND pin to the blue (minus) rail on the breadboard.

STARTER CODE

```
// Traffic light: green, then yellow, then red, forever
const int RED = 8;
const int YELLOW = 9;
const int GREEN = 10;

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  digitalWrite(GREEN, HIGH);   // go
  delay(4000);
  digitalWrite(GREEN, LOW);

  digitalWrite(YELLOW, HIGH);  // slow down
  delay(1500);
  digitalWrite(YELLOW, LOW);

  digitalWrite(RED, HIGH);     // stop
  delay(4000);
  digitalWrite(RED, LOW);
}
```

IF IT DOESN'T WORK

LED stays dark: it's probably in backwards. Flip it so the long leg faces the resistor.

Upload fails: wrong port selected, or a charge-only USB cable. Try another cable first.

Two LEDs light at once: a resistor or wire is in the same breadboard row as the wrong LED.

FINISHED EARLY?

- Add a second set of three LEDs for the cross street. When one direction is green, the other must be red.
- Add a "walk" signal: a white LED that only turns on while both directions are red.
- Make yellow blink three times before turning red.

SESSION 2 | 55 MINUTES

REACTION TIME GAME

Students build a game: the LED turns on after a random wait, and the Arduino measures how fast you hit the button. A quick reaction plays a high "win" tone on the buzzer. This is where code starts responding to the real world instead of just running a script, and it gets competitive fast.

PARTS PER PAIR

PART	QTY	NOTES
Arduino Uno + USB cable	1	
Half-size breadboard	1	
LED + 220 ohm resistor	1 each	Any color
Push button (4 leg tactile)	1	No resistor needed, we use INPUT_PULLUP
Piezo buzzer (passive)	1	Passive buzzers can play different pitches
Jumper wires	6	

RUN OF SESSION

TIME	WHAT HAPPENS	TEACHING NOTES
0 to 8 min	Plug in and upload Blink as a test	Same check as session 1. New students may need help picking the port.
8 to 18 min	Wire the LED to pin 9	Pin 9 to resistor to long leg, short leg to GND.
18 to 30 min	Wire the button to pin 2 and GND	Button straddles the center gap. One leg to pin 2, the diagonal leg to GND.
30 to 50 min	Build the game: random wait, LED on, time the press, play a tone	Have them open the Serial Monitor at 9600 baud to see their times.
50 to 55 min	Class tournament, then clean up	Fastest reaction in the room wins bragging rights.

Why no resistor on the button? INPUT_PULLUP turns on a resistor that's built into the Arduino. The pin reads HIGH normally and LOW when the button is pressed. It feels backwards at first, so say it out loud: pressed means LOW.

STARTER CODE

```
// Reaction game: wait for the light, then press as fast as you can
const int LED = 9;
const int BUTTON = 2;
const int BUZZER = 8;

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);   // pressed = LOW
  pinMode(BUZZER, OUTPUT);
  Serial.begin(9600);
  randomSeed(analogRead(A0));      // different random waits every time
}

void loop() {
  Serial.println("Get ready...");
  delay(random(2000, 5000));       // wait 2 to 5 seconds

  digitalWrite(LED, HIGH);
  unsigned long start = millis();
  while (digitalRead(BUTTON) == HIGH) {
    // wait here until the button is pressed
  }
  unsigned long reaction = millis() - start;
  digitalWrite(LED, LOW);

  Serial.print("Your time: ");
  Serial.print(reaction);
  Serial.println(" ms");

  if (reaction < 300) {
    tone(BUZZER, 1200, 300);       // fast: high tone
  } else {
    tone(BUZZER, 500, 300);        // slow: low tone
  }
  delay(2000);
}
```

IF IT DOESN'T WORK

Game finishes instantly with a time of 0: the button is wired across the wrong pair of legs, so it always reads pressed. Rotate it 90 degrees.

No sound: check the buzzer's + leg goes to pin 8. An active buzzer only plays one pitch, which is fine.

Serial Monitor shows junk characters: set the baud rate to 9600.

MODS MENU FOR FAST FINISHERS

- Play 3 rounds and print the average time.
- "Too early" penalty: if the button is already pressed when the LED comes on, play a buzz and restart.
- Make it harder every round by shrinking the 300 ms win threshold.
- Two player mode: a second button on pin 3, first press wins.

SESSION 3 | 70 MINUTES

SERVO CREATURE

Students attach a small servo motor to a cardboard creature and decide how it moves. A button steps through preset poses, or a knob (potentiometer) gives smooth control. This is the first taste of robotics: code that makes something physically move, and design choices about how it should feel.

PARTS PER PAIR

PART	QTY	NOTES
Arduino Uno + USB cable	1	
Half-size breadboard	1	
SG90 micro servo with horns	1	Brown = GND, red = 5V, orange = signal
Push button OR 10k potentiometer	1	Pairs choose one
Jumper wires	6	
Cardboard, foam, tape, markers, googly eyes	Shared	Set out a craft table

RUN OF SESSION

TIME	WHAT HAPPENS	TEACHING NOTES
0 to 10 min	Upload the Sweep example (File, Examples, Servo, Sweep)	Proves the servo and wiring work before anyone writes code.
10 to 25 min	Wire the SG90	Brown to GND, red to 5V, orange to pin 9.
25 to 35 min	Pick a control and wire it	Button: pin 2 and GND. Knob: middle leg to A0, outer legs to 5V and GND.
35 to 50 min	Code the behavior	Button steps through poses. Knob maps position to angle.
50 to 70 min	Build the creature and tune the motion	Tape the servo horn to a jaw, wing, or head. Adjust angles until it looks alive.

STARTER CODE: BUTTON

```
// Button version: each press moves the creature to its next pose
#include <Servo.h>

Servo creature;
const int BUTTON = 2;
int poses[] = {20, 90, 160};   // angles in degrees, change these
int pose = 0;

void setup() {
  creature.attach(9);
  pinMode(BUTTON, INPUT_PULLUP);
  creature.write(poses[pose]);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    pose = (pose + 1) % 3;           // 0, 1, 2, back to 0
    creature.write(poses[pose]);
    delay(250);
    while (digitalRead(BUTTON) == LOW) { }   // wait for release
  }
}
```

STARTER CODE: KNOB

```
// Knob version: turn the potentiometer, the creature follows
#include <Servo.h>

Servo creature;

void setup() {
  creature.attach(9);
}

void loop() {
  int knob = analogRead(A0);              // 0 to 1023
  int angle = map(knob, 0, 1023, 0, 180); // convert to 0 to 180 degrees
  creature.write(angle);
  delay(15);
}
```

IF IT DOESN'T WORK

Servo twitches or the Arduino resets: the servo is pulling too much power. Make sure nothing is blocking the horn, and don't run more than one servo off a laptop USB port.

Servo buzzes at the end of its travel: it's being pushed past its limit. Keep angles between about 10 and 170.

Knob only works over part of its turn: the outer legs are swapped or one isn't connected.

IDEAS FOR CREATURES

- A crocodile whose jaw chomps on each button press.
- A bird that flaps faster the further you turn the knob (hint: change the delay, not the angle).
- A nodding head that says yes or no to questions.

LOGISTICS AND SAFETY

Target grades	5 to 8. Younger students can do sessions 1 and 3 with more adult help.
Group size	As many as your room and helpers can handle, working in pairs. One helper per 5 or 6 pairs works well.
Session length	55 to 70 minutes each. They can run on three days in a row or once a week.
Safety	Low voltage electronics only (5 volts from USB). No soldering, no power tools, no batteries over 9V. Remind students not to bend LED legs back and forth, they snap.
Cleanup	Every part goes back in its labeled bag. Count LEDs and servos before students leave.

QUESTIONS? TALK TO A HUMAN.

Email sparkgearsinitiative@gmail.com or call 571-429-9220. Request a mentor and a supply kit at sparkgearsinitiative.org/for-teams.html. We usually reply within two school days.